

Estruturas de Dados — Vetor Array e Vetor Duplamente Encadeado

Vetor Array

TestesVetor.java

```
package Implementação.Vetor.Array;

public class TestesVetor {
    public static void main(String[] args){

        VetorArray check = new VetorArray(10);

        check.insertAtRank(0, 5);
        check.insertAtRank(1, 6);
        check.insertAtRank(2, 7);

        System.out.println(check.size());

        check.removeAtRank(1);

        System.out.println(check.size());

        check.replaceAtRank(1, 10);

        System.out.println(check.elemAtRank(1));
    }
}
```

Vetor.java

```
package Implementação.Vetor.Array;

public interface Vetor {

    int size();

    boolean isEmpty();

    Object elemAtRank(int r) throws VetorVazia;
```

```

    Object replaceAtRank(int r, Object o) throws VetorVazia;

    void insertAtRank(int r, Object o);

    Object removeAtRank(int r) throws VetorVazia;

    void grow();
}

```

VetorArray.java

```

package Implementação.Vetor.Array;

public class VetorArray implements Vetor{

    private Object[] arr;
    private int capacidade, size;

    public VetorArray(int capacidade){

        this.capacidade = capacidade;
        this.size = 0;

        arr = new Object[capacidade];
    }

    public Object elemAtRank(int r) throws VetorVazia{

        if (isEmpty()){
            throw new VetorVazia("Vetor array está vazia");
        }

        return arr[r];
    }

    public Object replaceAtRank(int r, Object o) throws VetorVazia{

        if (isEmpty()){
            throw new VetorVazia("Vetor array está vazia");
        }

        Object substitui = arr[r];

        arr[r] = o;
    }
}

```

```

        return substitui;
    }

    public void insertAtRank(int r, Object o){

        if (capacidade == size()){
            grow();
        }

        for (int i = size(); i > r; i--){
            arr[i] = arr[i - 1];
        }

        arr[r] = o;

        size++;
    }

    public Object removeAtRank(int r) throws VetorVazia{

        if (isEmpty()){
            throw new VetorVazia("Vetor array está vazia");
        }

        Object removi = arr[r];

        for (int i = r; i < size() - 1; i++){
            arr[i] = arr[i + 1];
        }

        size--;

        return removi;
    }

    public int size(){
        return size;
    }

    public boolean isEmpty(){
        return size == 0;
    }

    public void grow(){

```

```

        int nova_capacidade = capacidade * 2;

        Object[] novo_arr = new Object[nova_capacidade];

        for (int i = 0; i < size(); i++){
            novo_arr[i] = arr[i];
        }

        capacidade = nova_capacidade;

        arr = novo_arr;
    }
}

```

VetorVazia.java

```

package Implementação.Vetor.Array;

public class VetorVazia extends RuntimeException{

    public VetorVazia(String err){
        super(err);
    }
}

```

Vetor Duplamente Encadeado

No.java

```

package Implementação.Vetor.Duplamente;

public class No {

    private Object elemento;
    private No proximo, anterior;

    public void setAnterior(No anterior){
        this.anterior = anterior;
    }

    public void setProximo(No proximo){
        this.proximo = proximo;
    }
}

```

```

    }

    public void setElement(Object elemento){
        this.elemento = elemento;
    }

    public Object getElement(){
        return elemento;
    }

    public No getAnterior(){
        return anterior;
    }

    public No getProximo(){
        return proximo;
    }
}

```

TestesVetor.java

```

package Implementação.Vetor.Duplamente;

public class TestesVetor {

    public static void main(String[] args){

        VetorDupla vetor = new VetorDupla();

        vetor.insertAtRank(0, "A");
        vetor.insertAtRank(1, "B");
        vetor.insertAtRank(2, "C");

        System.out.println(vetor.elemAtRank(0));
        System.out.println(vetor.elemAtRank(1));
        System.out.println(vetor.elemAtRank(2));

        System.out.println("Removido: " + vetor.removeAtRank(1));

        System.out.println(vetor.elemAtRank(0));
        System.out.println(vetor.elemAtRank(1));

        vetor.replaceAtRank(1, "X");
    }
}

```

```

        System.out.println(vetor.elemAtRank(1));

        System.out.println("Tamanho: " + vetor.size());
    }
}

```

Vetor.java

```

package Implementação.Vetor.Duplamente;

public interface Vetor {

    int size();

    boolean isEmpty();

    Object elemAtRank(int r) throws VetorVazia;

    Object replaceAtRank(int r, Object o) throws VetorVazia;

    void insertAtRank(int r, Object o);

    Object removeAtRank(int r) throws VetorVazia;
}

```

VetorDupla.java

```

package Implementação.Vetor.Duplamente;

public class VetorDupla implements Vetor{

    private No head, tail;
    private int size;

    public VetorDupla(){

        this.head = new No();
        this.tail = new No();

        this.head.setElement(null);
        this.tail.setElement(null);

        this.head.setProximo(tail);
    }
}

```

```

        this.tail.setAnterior(head);

        this.size = 0;
    }

    public int size(){
        return size;
    }

    public void insertAtRank(int r, Object o){

        No anterior = head;

        if (r < 0 || r > size()){
            throw new VetorVazia("Indice invalido");
        }

        for (int i = 0; i < r; i++){
            anterior = anterior.getProximo();
        }

        No novo = new No();

        novo.setElement(o);

        No proximo = anterior.getProximo();

        novo.setAnterior(anterior);
        novo.setProximo(proximo);

        anterior.setProximo(novo);
        proximo.setAnterior(novo);

        size++;
    }

    public boolean isEmpty(){
        return size == 0;
    }

    public Object elemAtRank(int r) throws VetorVazia{

        if (isEmpty()){
            throw new VetorVazia("Vetor está vazia!");
        }
    }

```

```

        if (r < 0 || r >= size()){
            throw new VetorVazia("Indice invalido");
        }

        No elemento = head.getProximo();

        for (int i = 0; i < r; i++){
            elemento = elemento.getProximo();
        }

        return elemento.getElement();
    }

    public Object replaceAtRank(int r, Object o) throws VetorVazia{

        if (isEmpty()){
            throw new VetorVazia("Vetor está vazia!");
        }

        if (r < 0 || r >= size()){
            throw new VetorVazia("Indice invalido");
        }

        No elemento = head.getProximo();

        for (int i = 0; i < r; i++){
            elemento = elemento.getProximo();
        }

        Object salvo = elemento.getElement();

        elemento.setElement(o);

        return salvo;
    }

    public Object removeAtRank(int r) throws VetorVazia{

        if (isEmpty()){
            throw new VetorVazia("Vetor está vazia!");
        }

        if (r < 0 || r >= size()){
            throw new VetorVazia("Indice invalido");
        }
    }

```

```

        No elemento = head.getProximo();

        for (int i = 0; i < r; i++){
            elemento = elemento.getProximo();
        }

        No anterior = elemento.getAnterior();
        No proximo = elemento.getProximo();

        anterior.setProximo(proximo);
        proximo.setAnterior(anterior);

        size--;

        return elemento.getElement();
    }
}

```

VetorVazia.java

```

package Implementação.Vetor.Duplamente;

public class VetorVazia extends RuntimeException{

    public VetorVazia(String err){
        super(err);
    }
}

```